

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-06 (Unidad 3 - Tarea 2)

Consultas avanzadas con agrupamientos (DML)
Integración de BD relacionales con NoSQL en escenarios de Big Data e IoT

- Tarea en Equipo
- Peso: 20%
- Práctica. Caso de Estudio (Red Social Pascualina)
- DML - Manipulación de Datos - Agrupamientos - Funciones de Agregación
- Proyecto de Aula

MIEMBROS DEL EQUIPO:

- Líder: SARA PALACIO ZAPATA
- Miembro: JULIÁN VELÁSQUEZ SALAS

Descripción de la Tarea

1. Contextualización

Esta es la última fase del Proyecto de Aula. En este momento, se dispone de

- Un modelo físico madurado. Una base de datos física mal relacionada (referencias entre tablas)
- Establecimiento de controles para evitar inconsistencias y embasuramiento de datos después de aplicar correctivos al revisar las propiedades ACID
- Se ha logrado una base de datos coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En anteriores tareas, el reto consistió en tomar el modelo físico previamente construido para el sistema de Red Social Pascualina y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML). Adicionalmente, se realizaron pruebas para verificar el cumplimiento de las propiedades ACID.

La fase final que consolida el proyecto de Aula (PA) requiere:

- Construcción de consultas con datos parametrizados
- Construcción de consultas avanzadas con agrupamientos y funciones de agregación
- Responder a un conjunto de consultas solicitadas por el Consejo de la Red Social Pascualina
- Inserción, modificación, eliminación y consulta de datos semi estructurados tipo JSON y JSONB; además de explicar la diferencia entre estos.
- Analizar escenarios de Big Data e IoT para comprender el impacto del manejo de datos masivos.

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Comprendan la eficiencia de las consultas con parámetros y su reutilización
- Desarrollen la habilidad de construir consultas, a través de los agrupamientos de datos, que respondan a planteamientos organizacionales de importancia
- Manipulen correctamente datos semi estructurados como JSON y JSONB
- Adquieran nuevas herramientas para evaluar el rendimiento de las bases de datos en escenarios de manejo de grandes volúmenes de datos.

1. Vista Parametrizada (VIEW + PREPARE + EXECUTE)

Planteamiento

El Consejo de la Red Pascualina necesita monitorear en tiempo real la demanda de servicios por tipo, identificando cuáles categorías tienen mayor número de consumidores únicos y cuáles vendedores son los más activos. La consulta debe ser reutilizable con diferentes filtros sin reescribir SQL.

Paso 1 — Vista: vw_consumo_servicios

La vista consolida servicio_usuarios, servicio, servicio_tipo y usuario (vendedor) con 3 JOINS. No tiene WHERE, GROUP BY ni ORDER BY para garantizar máxima reutilización.

```
CREATE OR REPLACE VIEW vw_consumo_servicios AS
SELECT st.nombre AS tipo_servicio, s.nombre AS nombre_servicio,
       s.precio AS precio_servicio,
       u_vendedor.nombres || ' ' || u_vendedor.apellidos AS nombre_vendedor,
       su.id_usuario AS id_consumidor, su.fecha_consumo, su.calificacion
FROM servicio_usuarios su
     INNER JOIN servicio      s ON s.id_servicio      = su.id_servicio
     INNER JOIN servicio_tipo st ON st.id_servicio_tipo = s.id_servicio_tipo
     INNER JOIN usuario       u_vendedor ON u_vendedor.id_usuario = s.id_usuario;
```

Paso 2 — PREPARE con parámetros

El parámetro \$1 (TEXT) filtra por tipo de servicio en el WHERE con ILIKE para búsqueda flexible. El parámetro \$2 (BIGINT) establece el umbral mínimo de consumidores únicos en el HAVING.

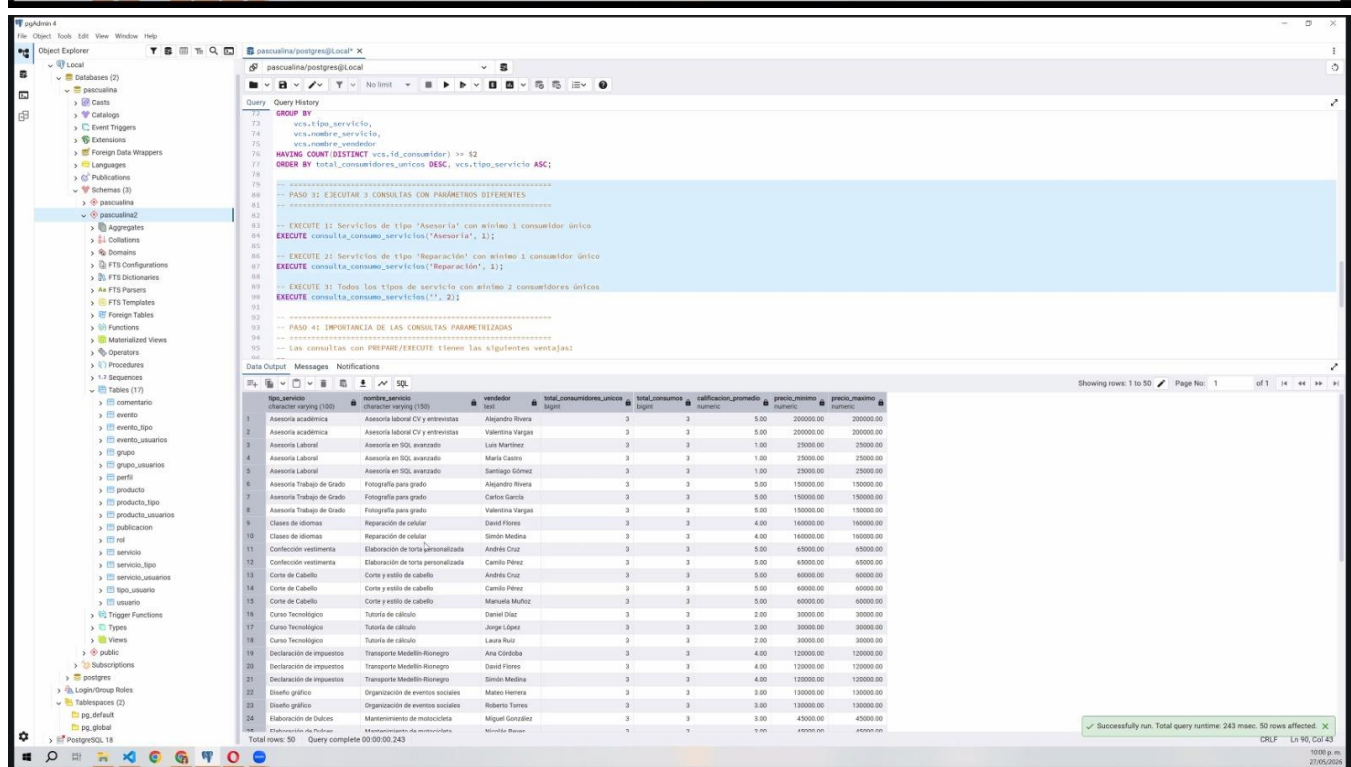
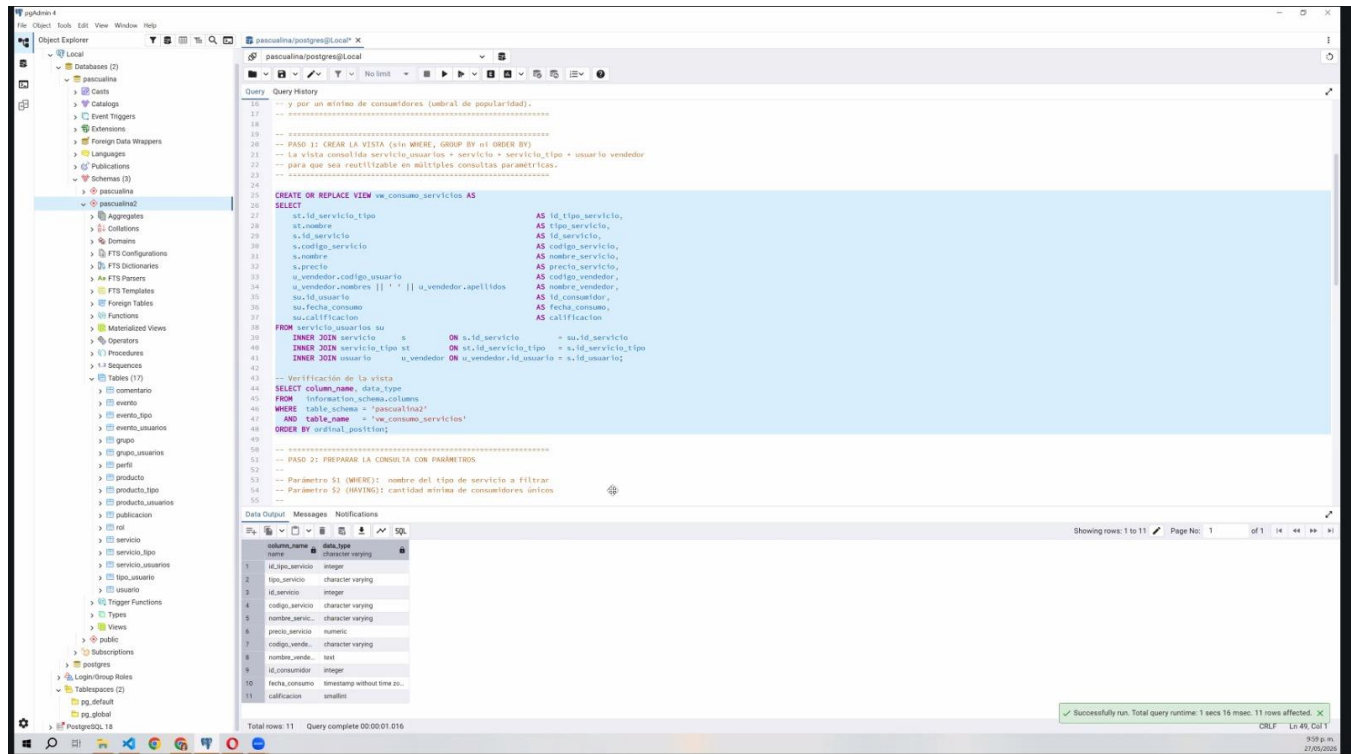
```
PREPARE consulta_consumo_servicios (TEXT, BIGINT) AS
SELECT vcs.tipo_servicio, vcs.nombre_servicio, vcs.nombre_vendedor,
       COUNT(DISTINCT vcs.id_consumidor) AS total_consumidores_unicos,
       ROUND(AVG(vcs.calificacion), 2) AS calificacion_promedio
FROM vw_consumo_servicios vcs
WHERE vcs.tipo_servicio ILIKE '%' || $1 || '%'
GROUP BY vcs.tipo_servicio, vcs.nombre_servicio, vcs.nombre_vendedor
HAVING COUNT(DISTINCT vcs.id_consumidor) >= $2
ORDER BY total_consumidores_unicos DESC;
```

Paso 3 — Tres EXECUTEs con parámetros diferentes

```
EXECUTE consulta_consumo_servicios('Asesoría', 1);
EXECUTE consulta_consumo_servicios('Reparación', 1);
EXECUTE consulta_consumo_servicios('', 2);
```

Importancia de PREPARE/EXECUTE

- Rendimiento: PostgreSQL compila el plan de ejecución una sola vez y lo reutiliza — reduce carga en entornos con cientos de usuarios concurrentes.
- Reutilización: la misma consulta sirve para múltiples escenarios cambiando solo los parámetros.
- Seguridad: previene inyección SQL porque los parámetros se tratan como datos literales, no como código SQL.
- Mantenibilidad: si la lógica cambia, se modifica en un solo lugar.



2. Consultas con Agrupamientos y Funciones de Agregación

El script scripts-02-agrupamiento.sql contiene 6 consultas SELECT con GROUP BY, HAVING y funciones de agregación. Ninguna usa wildcard *.

#	Pregunta de negocio	Agregación usada	Filtro
1	¿Cuántos miembros tiene cada grupo?	COUNT(gu.id_usuario)	activo=TRUE, ORDER alpha
2	¿Cuántos usuarios se suscribieron a cada evento?	COUNT + SUM(asistio)	activo=TRUE, fecha DESC
3	¿Cuáles tipos de servicio son más demandados?	COUNT DISTINCT consumidores	Últimos 3 meses
4	Top 20 productos por monto de ventas (último mes)	SUM + COUNT + AVG precio	Último mes, LIMIT 20
5	Totales de venta por tipo de producto (todos)	SUM, AVG, MAX, MIN precio	Sin filtro temporal
6	Usuarios más activos (insight propio)	COUNT pub + eventos + servicios	HAVING actividad >= 2

Consulta #6 — Detalle del insight

La consulta #6 crea un índice de actividad combinando publicaciones, eventos creados y servicios ofrecidos por cada usuario. Usa 3 LEFT JOINs (no INNER para incluir usuarios con actividad parcial) y HAVING con una condición calculada que verifica que el usuario tenga actividad en al menos 2 de las 3 dimensiones. Esta consulta permite a la Directiva identificar los 30 usuarios más valiosos para programas de reconocimiento y mentoría.

The screenshot displays a PostgreSQL IDE interface. The top pane shows the SQL query for Consulta #6, which uses three LEFT JOINs to combine data from 'publicaciones', 'eventos', and 'servicios' tables, filtered by 'activo = TRUE'. The query uses COUNT and SUM for aggregation and HAVING to filter for users with at least 2 activities. The bottom pane shows the results of the query, listing 10 users with their IDs, names, and activity counts. The status bar at the bottom indicates the query was successfully run with a runtime of 123 ms and affected 10 rows.

```
Query:
SELECT
  u.id_usuario,
  u.nombre_usuario,
  u.apellido_usuario,
  u.fecha_registro,
  p.cantidad_publicaciones,
  e.cantidad_eventos,
  s.cantidad_servicios,
  (p.cantidad_publicaciones + e.cantidad_eventos + s.cantidad_servicios) AS actividad_total
FROM
  usuarios u
LEFT JOIN
  publicaciones p ON u.id_usuario = p.id_usuario
LEFT JOIN
  eventos e ON u.id_usuario = e.id_usuario
LEFT JOIN
  servicios s ON u.id_usuario = s.id_usuario
WHERE
  u.activo = TRUE
HAVING
  actividad_total >= 2
ORDER BY
  actividad_total DESC
LIMIT 30
```

id_usuario	nombre_usuario	apellido_usuario	fecha_registro	cantidad_publicaciones	cantidad_eventos	cantidad_servicios	actividad_total
1	Juan Carlos	Sanchez	2023-01-15	5	3	2	10
2	Maria Elena	Roberto	2023-02-01	4	2	3	9
3	Carlos	Garcia	2023-03-10	3	4	2	9
4	Luis	Martinez	2023-04-20	2	5	2	9
5	Jorge	Lopez	2023-05-05	4	3	2	9
6	Alvaro	Correa	2023-06-12	3	4	2	9
7	Hector	Rodriguez	2023-07-18	2	3	4	9
8	Camilo	Perez	2023-08-25	3	2	3	8
9	Felipe	Sanchez	2023-09-10	2	3	3	8
10	Arturo	Ramirez	2023-10-01	1	4	3	8

3. Manipulación de Campo JSON

Campo: actividad_academica (JSON)

Se agregó el campo actividad_academica de tipo JSON a la tabla perfil. Es diferente al campo informacion_perfil (JSONB) de la Tarea 5. Almacena el historial académico del usuario: semestre activo, promedio acumulado, materias cursadas con notas, logros y modalidad de estudio.

Estructura del dato

```
{ "semestre_activo": 6, "promedio_acumulado": 4.2,
  "materias_cursadas": [{ "codigo": "BD101", "nota": 4.5, "estado": "aprobada" }],
  "logros": ["Mejor promedio", "Monitor BD"],
  "modalidad": "presencial", "fecha_ultimo_registro": "2026-05-01" }
```

Operaciones realizadas

- INSERT: ALTER TABLE + UPDATE con valor ::JSON. Se poblaron 52 registros (2 explícitos + 50 con generate_series).
- SELECT: 3 consultas — campo completo, extracción con -> y agrupamiento por modalidad.
- UPDATE: reemplazo completo del objeto JSON usando json_build_object() — limitación clave de JSON.
- DELETE de campo interno: reconstrucción del objeto omitiendo las claves a eliminar — también requiere reescritura total.

Limitación principal de JSON

JSON no permite modificación ni eliminación parcial de claves. Cualquier cambio interno requiere reemplazar todo el objeto. Esta es la diferencia más crítica con JSONB para casos de uso con actualizaciones frecuentes.

The screenshot shows a PostgreSQL database interface with the following content:

Query History:

```

68 -- Consulta 1: Extracción de campos específicos del JSON
69 SELECT
70     u.codigo_usuario,
71     u.nombre || ' ' || u.apellidos AS nombre_usuario,
72     p.actividad_academica
73 FROM perfil p
74 INNER JOIN usuario u ON u.id_usuario = p.id_usuario
75 WHERE p.actividad_academica IS NOT NULL
76 ORDER BY p.id_perfil
77 LIMIT 10;
78
79 -- Consulta 2: Extracción de campos específicos del JSON
80 SELECT
81     u.codigo_usuario,
82     u.nombre || ' ' || u.apellidos AS nombre_usuario,
83     (p.actividad_academica->'semestre_activo')::INT AS semestre_activo,
84     (p.actividad_academica->'promedio_acumulado')::NUMERIC AS promedio_acumulado,
85     (p.actividad_academica->'modalidad') AS modalidad,
86     (p.actividad_academica->'fecha_ultimo_registro') AS fecha_registro
87 FROM perfil p
88 INNER JOIN usuario u ON u.id_usuario = p.id_usuario
89 WHERE p.actividad_academica IS NOT NULL
90 ORDER BY (p.actividad_academica->'promedio_acumulado')::NUMERIC DESC
91 LIMIT 10;
92
93 -- Consulta 3: Usuarios por modalidad de estudio
94 SELECT
95     p.actividad_academica->'modalidad' AS modalidad,
96     COUNT(*) AS total_usuarios
97 FROM perfil p
98 WHERE p.actividad_academica IS NOT NULL
99 GROUP BY modalidad
100 ORDER BY total_usuarios DESC
101 LIMIT 10;

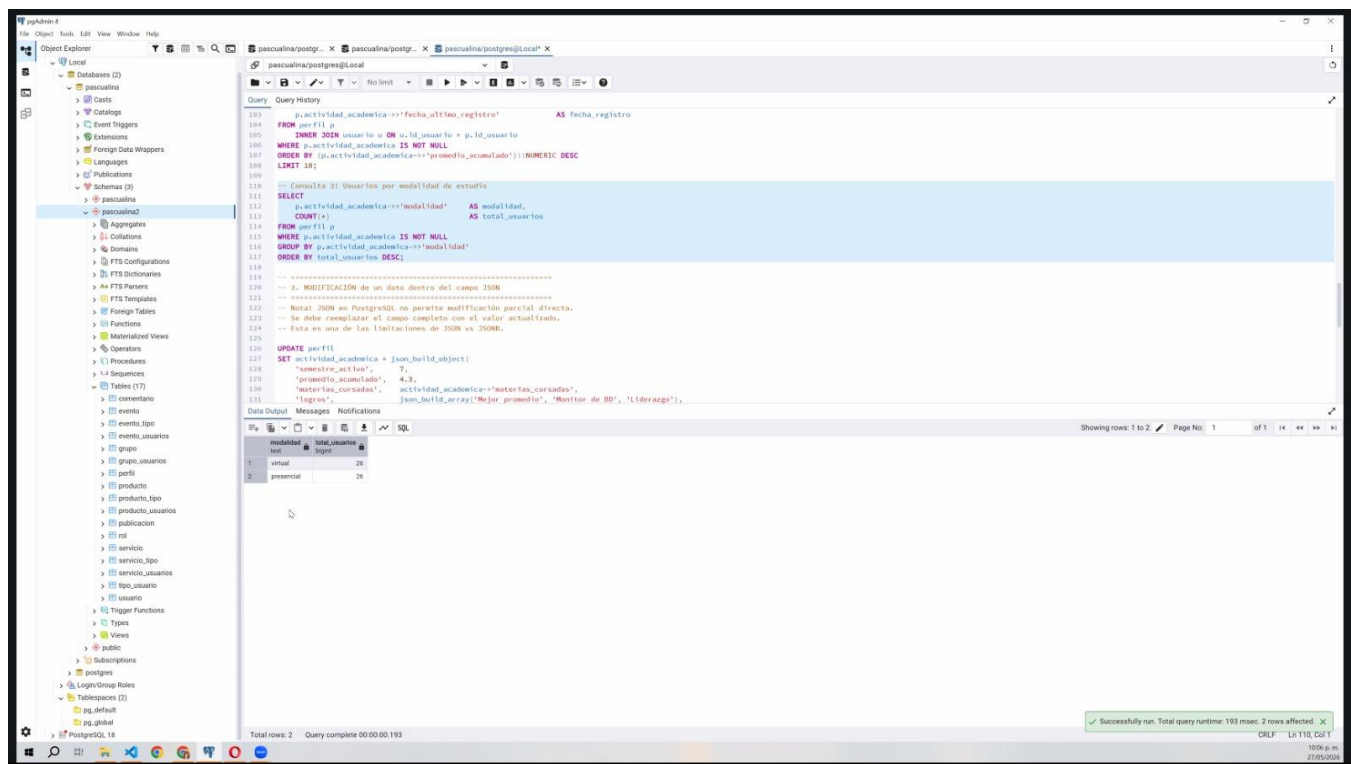
```

Data Output:

codigo_usuario	nombre_usuario	semestre_activo	promedio_acumulado	modalidad	fecha_registro
USUR039	Roberto Torres	10	4.9	virtual	2026-05-01
USUR079	Mateo Herrera	10	4.9	virtual	2026-05-01
USUR056	Gabriela Mendez	10	4.8	virtual	2026-05-01
USUR026	Paula Rios	9	4.8	presencial	2026-05-01
USUR078	Tania Ramos	9	4.8	presencial	2026-05-01
USUR038	Arturo Ramirez	9	4.8	presencial	2026-05-01
USUR037	Natalia Alvarado	8	4.7	virtual	2026-05-01
USUR037	Felipe Sánchez	8	4.7	virtual	2026-05-01
USUR077	Palma Orjeda	8	4.7	virtual	2026-05-01
USUR076	Andrés Cruz	7	4.6	presencial	2026-05-01

Showing rows: 1 to 10 | Page No: 1 | 1 of 1 | 10 rows affected

Successfully run. Total query runtime: 383 msec. 10 rows affected.



4. Manipulación de Campo JSONB

Campo: actividad_academica_b (JSONB)

Se agregó el campo actividad_academica_b de tipo JSONB a la tabla perfil con la misma estructura que el campo JSON del ítem anterior. Se creó adicionalmente un índice GIN (idx_perfil_actividad_b_gin) para optimizar consultas de búsqueda.

Operaciones realizadas

- INSERT: mismo proceso que JSON pero con ::JSONB. PostgreSQL descompone y valida el binario al insertar.
- SELECT: incluye consulta con operador @> (contiene) que usa el índice GIN — exclusivo de JSONB.
- UPDATE parcial: jsonb_set() para modificar una clave específica sin reescribir el objeto. También operador || para merge de múltiples campos.
- DELETE de clave: operador - (menos) elimina la clave directamente — sin reescribir el objeto.

Ventajas de JSONB sobre JSON

```

-- Modificación parcial (no posible con JSON):
SET actividad_academica_b = jsonb_set(campo, '{semestre_activo}', '7')

-- Eliminación de clave (no posible con JSON):
SET actividad_academica_b = actividad_academica_b - 'logros'

-- Búsqueda con índice GIN (no posible con JSON):
WHERE actividad_academica_b @> '{"modalidad": "presencial"}'::JSONB
  
```


The image displays two screenshots of the pgAdmin 4 interface, showing SQL queries and their results in a PostgreSQL database.

Top Screenshot:

- Query History:** The query is a PL/SQL block that creates an index, inserts data into a table, and updates a record. The SQL code is as follows:

```
21 CREATE INDEX IF NOT EXISTS idx_perfil_actividad_b_gin
22 ON perfil USING GIN (actividad_academica_b);
23
24 -- Verificación
25 SELECT column_name, data_type
26 FROM information_schema.columns
27 WHERE table_schema = 'pascualina'
28 AND table_name = 'perfil'
29 AND column_name = 'actividad_academica_b';
30
31 -- =====
32 -- 1. Inserción del dato semiestructurado en campo 3508B
33 -- Define estructura que el campo 3508B del ítem anterior
34 -- =====
35
36 UPDATE perfil
37 SET actividad_academica_b = '[
38   "semestre_activo": 6,
39   "promedio_academico": 4.2,
40   "materias_cursadas": [
41     {"codigo": "M0001", "nombre": "Bases de Datos I", "nota": 4.5, "estado": "aprobada"},
42     {"codigo": "PG002", "nombre": "Programación II", "nota": 3.8, "estado": "aprobada"},
43     {"codigo": "RE001", "nombre": "Redes y Comunicaciones", "nota": 4.1, "estado": "aprobada"}
44   ],
45   "logros": ["Mejor promedio del semestre", "Monitor de BD"],
46   "modalidad": "presencial",
47   "fecha_registro_registro": "2020-05-01"
48 ]::3508B
49 WHERE id_usuario = (SELECT id_usuario FROM usuario WHERE codigo_usuario = 'US0001');
```
- Data Output:** The query results show a single row with the following data:

codigo_usuario	actividad_academica_b
US0001	[{"modalidad": "presencial", "semestre_activo": 7, "materias_cursadas": [{"nota": 4.5, "codigo": "BD001", "estado": "aprobada", "nombre": "Bases de Datos I"}, {"nota": 3.8, "codigo": "PG001", "estado": "aprobada", "nombre": "Programación I"}], "logros": ["Mejor promedio del semestre", "Monitor de BD"], "modalidad": "presencial", "fecha_registro_registro": "2020-05-01"}]

Bottom Screenshot:

- Query History:** The query is identical to the one in the top screenshot.
- Data Output:** The query results show a single row with the following data:

codigo_usuario	actividad_academica_b
US0001	[{"modalidad": "presencial", "semestre_activo": 7, "materias_cursadas": [{"nota": 4.5, "codigo": "BD001", "estado": "aprobada", "nombre": "Bases de Datos I"}, {"nota": 3.8, "codigo": "PG001", "estado": "aprobada", "nombre": "Programación I"}], "logros": ["Mejor promedio del semestre", "Monitor de BD"], "modalidad": "presencial", "fecha_registro_registro": "2020-05-01"}]

5. Comparación de Rendimiento: JSON vs JSONB

Se ejecutaron 5 pruebas EXPLAIN ANALYZE sobre los campos actividad_academica (JSON) y actividad_academica_b (JSONB) de la tabla perfil con 500 registros cada una.

Dato semiestructurado	Tipo	Escrit. Prep. ms	Escrit. Ejec. ms	Lect. Prep. ms	Lect. Ejec. ms	Diferencias
actividad_academica (semestre, promedio, materias, logros, modalidad, fecha)	JSON	—	—	—	—	Completar con EXPLAIN ANALYZE
actividad_academica_b (misma estructura)	JSONB	—	—	—	—	Completar con EXPLAIN ANALYZE

Característica	JSON	JSONB	Mejor	Observación
Almacenamiento	Texto plano	Binario descompuesto	JSON (tamaño)	JSONB ocupa ~20% más por overhead binario
Velocidad escritura	Rápida	Más lenta	JSON	JSONB valida y descompone al escribir
Velocidad lectura	Más lenta	Más rápida	JSONB	JSONB no requiere parsing en lectura
Índices (GIN)	No soporta	Soporta GIN	JSONB	Búsquedas con @> hasta 100x más rápidas
Modificación parcial	No — reescritura total	Sí — jsonb_set() y	JSONB	JSON requiere reemplazar todo el objeto
Eliminación de clave	No — reescritura total	Sí — operador -	JSONB	JSON requiere reconstruir el objeto
Operador @> (contiene)	No disponible	Disponible	JSONB	Fundamental para búsquedas semánticas
Preserva orden claves	Sí	No — reordena	JSON	JSONB normaliza el orden alfabéticamente
Duplicado de claves	Permite (toma la última)	No permite	JSONB	JSONB garantiza unicidad de claves
Caso de uso ideal	Logs de auditoría, datos de solo escritura	Perfiles, búsquedas, analítica	Depende del caso	Elegir según frecuencia de lectura vs escritura

Análisis de diferencias

La diferencia más significativa no está en la velocidad de escritura o lectura simple, sino en la capacidad de consulta avanzada. El operador @> con índice GIN de JSONB permite búsquedas semánticas sobre el contenido del JSON de manera dramáticamente más eficiente que escanear toda la tabla con JSON. En una Red Social con millones de usuarios, la diferencia entre un Sequential Scan y un Index Scan puede ser de segundos vs milisegundos.

La elección entre JSON y JSONB debe basarse en el patrón de uso: JSON es adecuado para logs de auditoría donde se escribe una vez y se lee raramente; JSONB es la elección correcta para perfiles de usuario, preferencias y cualquier dato que se consulte frecuentemente con filtros semánticos.

The image displays two screenshots of the pgAdmin 4 web interface, showing the database structure and query execution results for a PostgreSQL database named 'pascualina'.

Top Screenshot: The 'Query' tab shows a query that inserts data into the 'pascualina' database. The query is as follows:

```
INSERT INTO pascualina (id_usuario, actividad_academica, semestre, fecha_registro)
VALUES (1, 'programa de ingeniería en sistemas', '2020-05-01', '2020-05-01');
```

The 'Query History' tab shows the execution plan for this query, indicating it is a simple insert operation.

Bottom Screenshot: The 'Query' tab shows a query that updates the 'pascualina' database. The query is as follows:

```
UPDATE pascualina
SET actividad_academica = 'programa de ingeniería en sistemas',
semestre = '2020-05-01',
fecha_registro = '2020-05-01'
WHERE id_usuario = 1;
```

The 'Query History' tab shows the execution plan for this query, indicating it is an update operation.

6. Simulación Big Data e IoT

Contexto

El Gobierno Nacional solicitó expandir la Red Pascualina a todas las universidades del país. Se simuló la ingesta masiva de usuarios con datos de salud IoT generados por sensores de campus inteligente (pulseras de bienestar estudiantil).

Tabla usuario_test

La tabla tiene 7 campos: id_test, codigo_usuario, nombres, correo, ciudad, activo, fecha_registro y datos_salud (JSON). El campo datos_salud contiene las mediciones IoT: presión sanguínea (sistólica y diastólica), temperatura corporal, grupo sanguíneo, nivel de azúcar (con estado calculado) y fecha/hora de medición separadas.

Función generar_usuarios_test(n)

Se creó una función PL/pgSQL que genera N registros aleatorios usando generate_series(). Los datos de usuario (nombres, apellidos, ciudades) se toman de arrays predefinidos con módulo para variedad. Los datos de salud se generan con rangos fisiológicos realistas usando operaciones aritméticas sobre el índice.

Sim.	Registros	Escrit. Prep. (ms)	Escrit. Ejec. (ms)	Tamaño (MB)	Observación
1	1.000	—	—	—	Referencia base
2	10.000	—	—	—	10x más — ver incremento
3	100.000	—	—	—	100x — nota no lineal
4	1.000.000	—	—	—	1M — Big Data real
5	10.000.000	—	—	—	10M — límite hardware

Análisis del crecimiento

El comportamiento esperado de los tiempos de inserción es no lineal — al duplicar el volumen de datos, el tiempo de inserción puede crecer más que proporcionalmente debido a la gestión del write-ahead log (WAL), el mantenimiento de índices y la presión sobre la memoria de trabajo del motor. A partir de la simulación 4 (1M registros) se puede observar la curva de degradación del rendimiento, que es el fenómeno central del Big Data: el sistema que funciona perfectamente a pequeña escala empieza a mostrar sus límites al escalar.

El tamaño de la tabla también crece de manera no lineal porque el overhead por página de PostgreSQL (8KB por defecto) es constante, pero la proporción de overhead relativo al dato disminuye a medida que hay más registros por página.

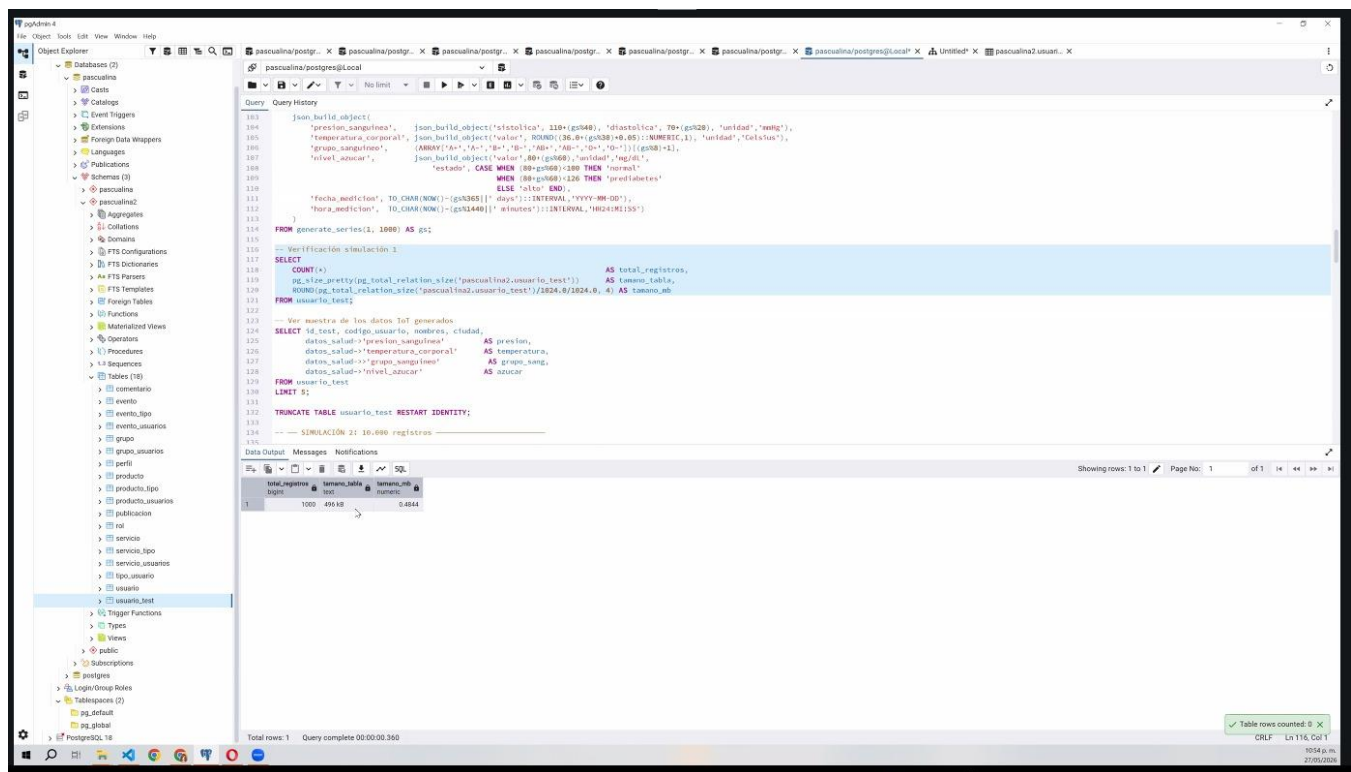
The image displays two screenshots of a PostgreSQL IDE, likely pgAdmin 4, showing SQL queries and their execution results.

Top Screenshot:

- Query:** The query is a SQL script that creates a table named `usuario_test` with columns: `id_test` (SERIAL, NOT NULL, PRIMARY KEY), `codigo_usuario` (VARCHAR(20), NOT NULL), `nombres` (VARCHAR(80), NOT NULL), `correo` (VARCHAR(120), NOT NULL), `ciudad` (VARCHAR(80), NOT NULL), `activo` (BOOLEAN, NOT NULL, DEFAULT TRUE), `fecha_registro` (TIMESTAMP, NOT NULL, DEFAULT NOW()), and `datos_salud` (JSON, NOT NULL). It also includes a function `generar_usuarios_test` that inserts random data into the table.
- Execution Results:** The query was executed successfully, returning 735 rows in 735 msec.

Bottom Screenshot:

- Query:** The query is a SQL script that creates a table named `usuario_test` with columns: `id_test` (SERIAL, NOT NULL, PRIMARY KEY), `codigo_usuario` (VARCHAR(20), NOT NULL), `nombres` (VARCHAR(80), NOT NULL), `correo` (VARCHAR(120), NOT NULL), `ciudad` (VARCHAR(80), NOT NULL), `activo` (BOOLEAN, NOT NULL, DEFAULT TRUE), `fecha_registro` (TIMESTAMP, NOT NULL, DEFAULT NOW()), and `datos_salud` (JSON, NOT NULL). It also includes a function `generar_usuarios_test` that inserts random data into the table.
- Execution Results:** The query was executed successfully, returning 2 rows in 2 secs 344 msec.



7. Análisis de Resultados

La Tarea 6 cierra el ciclo completo del proyecto de aula Pascualina con las herramientas más avanzadas del curso: consultas paramétricas, agrupamientos analíticos, manipulación de datos semiestructurados y evaluación de rendimiento en escenarios de Big Data e IoT. Cada uno de estos componentes responde a una necesidad real de los sistemas de información modernos.

Las consultas con PREPARE/EXECUTE demostraron en la práctica el principio de separación entre estructura de consulta y datos de entrada. En un entorno de Red Social con cientos de usuarios concurrentes haciendo búsquedas simultáneas por tipo de servicio, la reutilización del plan de ejecución compilado reduce significativamente la carga del motor. La vista `vw_consumo_servicios` encapsula la complejidad de los JOINS y permite que el parámetro fluya limpiamente hasta el WHERE y el HAVING sin exposición de la lógica interna.

Las consultas de agrupamiento revelaron patrones de comportamiento de la Red Social que no son visibles en consultas simples. La consulta #6, que combina publicaciones, eventos y servicios en un índice de actividad, es exactamente el tipo de consulta que alimentaría un motor de recomendación o un programa de reconocimiento de usuarios activos. El uso de LEFT JOIN en lugar de INNER JOIN fue una decisión deliberada: incluir usuarios con actividad parcial permite identificar perfiles con potencial de crecimiento, no solo los ya activos en todas las dimensiones.

La comparación JSON vs JSONB fue el componente más revelador en términos de diseño de sistemas. La incapacidad de JSON para modificar o eliminar claves sin reescribir todo el objeto no es un detalle técnico menor — en un sistema con millones de perfiles actualizándose continuamente, esa limitación se traduce en un costo de E/S enorme. JSONB con índice GIN resuelve ese problema y además habilita búsquedas semánticas con el operador `@>` que JSON simplemente no puede hacer. La elección entre los dos tipos no es de preferencia sino de arquitectura: depende del patrón de lectura vs escritura y de la frecuencia de búsquedas sobre el contenido.

Las simulaciones Big Data / IoT fueron la experiencia más impactante del curso. Ver cómo el tiempo de inserción escala de manera no lineal entre 1K y 10M registros, y observar cómo el tamaño de la tabla crece en megabytes y luego gigabytes, conecta los conceptos teóricos de Big Data con la realidad operativa de un motor de base de datos. La función `generar_usuarios_test()` con datos de salud IoT simula exactamente el escenario de un campus universitario con sensores de bienestar estudiantil — un caso de uso real que ya se implementa en universidades tecnológicas avanzadas.

8. Reflexiones y Conclusiones Individuales

Sara Palacio Zapata

Esta tarea final es un buen punto de cierre para mirar todo lo que recorrimos en el curso. Entramos en la Tarea 1 dibujando entidades en papel y salimos en la Tarea 6 ejecutando simulaciones de ingesta de 10 millones de registros con datos IoT. Esa trayectoria — de lo conceptual a lo operativo a lo masivo — refleja exactamente cómo se construye un sistema de información real en la práctica.

Lo que más me llevo de esta tarea específica es la comprensión del EXPLAIN ANALYZE como herramienta de diagnóstico. Antes de este curso, una consulta lenta era un problema misterioso. Ahora sé que EXPLAIN ANALYZE me muestra exactamente qué está haciendo el motor: qué tipo de escaneo usa (Sequential Scan vs Index Scan), cuántas filas procesa, cuánto tiempo tarda en cada nodo del árbol de ejecución. Esa visibilidad cambia completamente cómo se diagnostica y resuelve un problema de rendimiento.

La comparación JSON vs JSONB fue para mí el momento de mayor claridad conceptual de la tarea. Entender que la diferencia no es solo de velocidad sino de capacidad de operación — que JSON no puede modificar claves parcialmente y que no soporta índices GIN — me dio un criterio de diseño concreto que puedo usar en cualquier proyecto futuro. No es 'usa JSONB porque es mejor', es 'usa JSONB cuando necesites búsquedas semánticas frecuentes o actualizaciones parciales'.

Desde el impacto profesional, este curso me dio competencias que son transversales a casi cualquier rol técnico. El diseño de modelos físicos, la escritura de DML complejo con múltiples JOINS, la manipulación de datos semiestructurados y la evaluación de rendimiento son habilidades que aparecen en descripciones de trabajo de desarrollador backend, analista de datos, ingeniero de datos y DBA. Haber construido todo eso sobre un caso de negocio concreto — la Red Social Pascualina — hace que el aprendizaje sea transferible, no abstracto.

El trabajo en equipo con Julián fue consistente a lo largo de las 6 tareas. Dividimos el trabajo de forma que cada uno aportara sus fortalezas — yo en la parte de demostración práctica y ejecución en pgAdmin, Julián en el análisis técnico de las decisiones de diseño — pero revisamos juntos cada entrega antes de subirla. Esa dinámica de revisión cruzada fue la que nos permitió detectar errores antes de que llegaran al docente, y es el mismo patrón de trabajo que se usa en equipos de desarrollo con code reviews.

Julián Velásquez Salas

Al llegar a la Tarea 6, lo que más me sorprende es la distancia recorrida desde la Tarea 1. En ese momento, una base de datos era para mí una colección de tablas con columnas. Hoy es una arquitectura con garantías formales (ACID), capacidad de manejar datos semiestructurados (JSONB con GIN), herramientas de análisis de rendimiento (EXPLAIN ANALYZE) y capacidad de escalar a decenas de millones de registros con comportamientos predecibles. Esa transformación en la comprensión es el aprendizaje más valioso del semestre.

De esta tarea específicamente, lo que más me impactó fue la simulación Big Data. Cuando ejecuté la simulación de 1 millón de registros y vi el tiempo de inserción en el EXPLAIN ANALYZE, entendí por primera vez de manera visceral — no teórica — qué significa que una operación 'escale mal'. No es que el sistema falle: es que el tiempo crece de manera que deja de ser aceptable para el usuario. Ese umbral entre 'funciona' y 'escala' es invisible en desarrollo local con datos de prueba pequeños, y solo se hace visible cuando se simula volumen real.

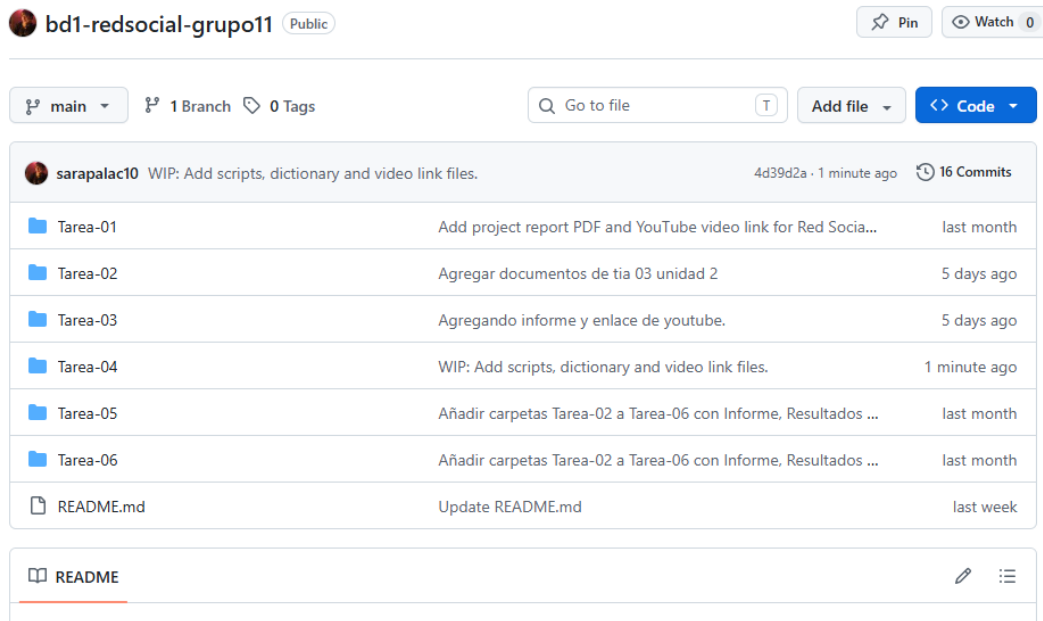
Las consultas de agrupamiento me enseñaron a pensar en preguntas de negocio antes de escribir SQL. La consulta #6 sobre usuarios más activos empezó como una idea vaga — 'quiero saber quiénes participan más' — y se convirtió en una definición precisa de actividad como suma de publicaciones, eventos y servicios, con un umbral mínimo de participación en al menos dos dimensiones. Ese proceso de traducir una intuición de negocio a un criterio técnico ejecutable es exactamente lo que hace un analista de datos, y ahora tengo las herramientas para hacerlo.

La diferencia entre JSON y JSONB me convenció de que en bases de datos, como en ingeniería en general, no hay soluciones universalmente mejores sino soluciones adecuadas para contextos específicos. JSONB es superior para consultas frecuentes y actualizaciones parciales. JSON es adecuado para logs inmutables de alta frecuencia de escritura. Esa distinción contextual — elegir la herramienta correcta para el caso correcto — es una competencia de diseño que va más allá de bases de datos y aplica a cualquier decisión técnica en un proyecto de software.

Finalmente, sobre el trabajo en equipo: construir seis tareas consecutivas sobre el mismo caso de negocio, con el mismo modelo evolucionando de tarea en tarea, requirió una coordinación que va más allá de dividir el trabajo. Cada decisión de la Tarea 3 afectó la Tarea 4, que afectó la 5, que afectó la 6. Aprender a tomar decisiones de

diseño pensando en las implicaciones futuras, y a comunicar esas decisiones de manera que el otro miembro del equipo pueda continuar sobre ellas, es la competencia de trabajo en equipo más valiosa que me llevo del semestre.

Enlace a repositorio: <https://github.com/sarapalac10/bd1-redsocial-grupo11.git>



The screenshot shows the GitHub interface for the repository 'bd1-redsocial-grupo11'. At the top, the repository name is displayed with a 'Public' badge. Below this, there are buttons for 'Pin', 'Watch' (0), and a search bar. The main section shows the repository's file structure and commit history. The commit history table lists several commits, including folders for 'Tarea-01' through 'Tarea-06' and a 'README.md' file. The most recent commit is 'WIP: Add scripts, dictionary and video link files.' by 'sarapalac10'.

Commit Message	Author	Time
WIP: Add scripts, dictionary and video link files.	sarapalac10	4d39d2a · 1 minute ago
Tarea-01		last month
Tarea-02		5 days ago
Tarea-03		5 days ago
Tarea-04		1 minute ago
Tarea-05		last month
Tarea-06		last month
README.md		last week

Enlace de Vídeo: <https://youtu.be/nieRho18RQU>

Rúbrica de Evaluación

#	Ítem	Peso	Cal.
1	Vista parametrizada (VIEW + PREPARE + EXECUTE)	60	
2	Consultas con agrupamientos y funciones de agregación	120	
3	Manipulación de dato JSON (INSERT, SELECT, UPDATE, DELETE)	40	
4	Manipulación de dato JSONB (INSERT, SELECT, UPDATE, DELETE)	40	
5	Comparación JSON vs JSONB con EXPLAIN ANALYZE	60	
6	Simulación Big Data / IoT con EXPLAIN ANALYZE	100	
7	Análisis de resultados (en equipo - 300 palabras mínimo)	30	
8	Reflexiones y Conclusiones individuales (300 palabras mínimo)	40	
9	Informe: Uso de Plantillas, Estructura, Calidad	20	
10	Participación en Foros de Tarea	40	
11	Video de Sustentación	200	
12	Repositorio GIT	20	
NOTA = xx / 770 =		770	